

JavaScript Fundamentals — Session Notes

Organized from your instructor session on [index.js](#). Sections marked  **Added by Claude** cover gaps that weren't in the original file but that you'll need soon — explained more at the bottom.

1. Output & Input Methods

These only work in a **browser environment** (not Node.js):

Method	Purpose	Returns
<code>alert("msg")</code>	Shows a popup dialog	<code>undefined</code>
<code>prompt("question")</code>	Popup with a text input	The string typed, or <code>null</code> if cancelled
<code>confirm("question")</code>	Popup with OK/Cancel	<code>true</code> or <code>false</code>
<code>document.write("msg")</code>	Writes directly into the HTML page	—
<code>console.log("msg")</code>	Prints to the browser/Node dev console	—

`console.log` is the one you'll use constantly for debugging; the others are mostly for learning purposes or simple UI prompts.

2. Comments

```
// single line comment

/* multi
  line
  comment */
```

3. Variables & Dynamic Typing

JavaScript is **loosely typed** (also called **dynamically typed**). You don't declare a data type — the type is decided automatically based on the value assigned, and it can change at runtime.

Compare to a statically typed language:

```
int num = 10;
String text = "hello";
boolean boolean = true;
```

```
var num = 10;
var text = "hello";
var boolean = true;
```

```
var _text = 2.5555255582145255;
console.log(typeof _text); // "number"
// same variable name reused with a string later would just become "string"
```

4. Variable Naming Rules

A variable name can contain:

- Letters **a-z**, **A-Z**
- Digits **0-9** (but **not** as the first character)
- Underscore **_**
- Dollar sign **\$**

```
var _text = "hello"; // valid
var $txt = "hello"; // valid
```

Names are case-sensitive, and by convention use **camelCase**.

5. **typeof** Operator

Returns a string naming the data type of a value:

```
console.log(typeof _text); // "string"
console.log(typeof $txt); // "string"
```

6. Primitive Data Types

JavaScript has 5 primitives covered in your notes (a 6th, **Symbol**, and a 7th, **BigInt**, exist but are more advanced — 

Type	Example	typeof result
number	10, 10.5, 55.4562586325	"number"
string	"5", 'text'	"string"
boolean	true, false	"boolean"

Type	Example	typeof result
undefined	<code>var txt;</code> (declared, no value)	"undefined"
null	<code>var x = null;</code>	"object" ⚠️

⚠️ **Important quirk** (NEW): `typeof null` returns "object", not "null". This is a famous bug baked into JavaScript since 1995 — it's kept for backward compatibility and will never be fixed. Just memorize it as an exception.

7. Object Types

a) Language (Wrapper) Objects

Created with `new`, wrapping a primitive value in an object:

```
var text = new String("Hello");
console.log(text);           // [String: 'Hello']
console.log(typeof text);    // "object"

var txt = "Hello";
console.log(txt);            // "Hello"
console.log(typeof txt);     // "string"
```

`new String()`, `new Boolean()`, `new Number()`, and `Array` fall into this category. In practice, you almost always want the plain literal ("Hello"), not the wrapped object — the wrapped version behaves unexpectedly in comparisons (NEW: e.g. `new String("a") === "a"` is `false`).

b) null

A primitive value representing "intentionally no object," though `typeof` misreports it as "object" (see above).

c) BOM — Browser Object Model

Objects the browser provides that represent the browser window itself (not part of the JS language spec):

- `Window`
- `Navigator`
- `History`
- `Screen`

d) DOM — Document Object Model

Represents the HTML page as a tree of objects you can read/manipulate (e.g.

`document.getElementById(...)`).

e) User-Defined Objects

1. **Object literal:** `var obj = { key: "value" };`
2. **Class-based:** using the `class` keyword and `new`

8. NaN — Not a Number

```
var x = Number("2t");  
console.log(x);           // NaN  
console.log(typeof x);    // "number" ← NaN is still typed as "number"
```

Key gotcha: NaN is never equal to anything, including itself:

```
console.log(NaN == NaN); // false  
console.log(NaN === NaN); // false
```

 **Added: checking for NaN properly.** Because `NaN === NaN` is false, you can't check for it with `===`. Use:

```
console.log(Number.isNaN(x)); // true – the reliable, modern way  
console.log(isNaN(x));        // true – older global function, but it coerces its  
argument first, which can give misleading results (e.g. isNaN("hello") is also  
true)
```

9. Equality Operators: `==` vs `===`

Operator	Compares	Type coercion?
<code>==</code>	values only	Yes
<code>===</code>	values and types	No

```
console.log("1" == 1); // true – string coerced to number first  
console.log("1" === 1); // false – different types  
  
console.log(undefined == null); // true – special-cased as "loosely equal"  
console.log(undefined === null); // false – different types  
  
console.log({} == {}); // false – objects compared by reference, not content  
console.log({} === {}); // false – same reason
```

 **Best practice:** Use `===` by default. It avoids the surprising coercion rules of `==` and is what virtually all modern style guides recommend.

10. Type Coercion Table

How JavaScript converts different values when you call `Number()`, `String()`, or `Boolean()` on them:

Value	Number()	String()	Boolean()
false	0	"false"	false
true	1	"true"	true
0	0	"0"	false
1	1	"1"	true
"0"	0	"0"	true
NaN	NaN	"NaN"	false
Infinity	Infinity	"Infinity"	true
-Infinity	-Infinity	"-Infinity"	true
"" (empty string)	0	""	false
"one"	NaN	"one"	true
[] (empty array)	0	""	true
[20]	20	"20"	true
[10, 20]	NaN	"10,20"	true
["txt"]	NaN	"txt"	true
[false]	NaN	"false"	true
function(){}	NaN	source code text	true
{}/[{}]	NaN	"[object Object]"	true
null	0	"null"	false
undefined	NaN	"undefined"	false

How to read the array rows: an array is first converted to a string by joining its items with commas (`[10, 20].toString()` → `"10,20"`), then that string is fed into `Number()` or kept as-is for `String()`. That's why `[20]` becomes the number `20` (a single item, no comma, cleanly parses) but `[10, 20]` becomes `NaN` (the comma makes it unparseable as a number).

The exercise at the end of your file, explained:

```
console.log(Number([{}])); // NaN
// [{}].toString() = "[object Object]", Number() of that = NaN
console.log(String([{}])); // "[object Object]"
// array joins to the object's string form
```

```
console.log(Boolean([{}])); // true
```

– ALL objects and arrays are truthy, no matter what's inside

The single most important rule to walk away with — Truthy vs. Falsy:

There are only **6 falsy values** in all of JavaScript:

```
false, 0, "", null, undefined, NaN
```

Everything else is truthy — **including [] and {}**, which trips up almost everyone at first, since they "look empty."

11. String Methods

Setup used throughout this section:

```
var text = "Hello";
```

a) Strings vs. String Objects

```
var text = "Hello";           // primitive string
var strObject = new String("Hello"); // String object (wrapper)

console.log(typeof text);      // "string"
console.log(typeof strObject); // "object"
```

Ties back to Section 7a — prefer the primitive literal. The object form causes surprises like `new String("Hello") === "Hello"` being `false`.

 **Strings are immutable.** No string method ever changes the original string — every one of them *returns a new string*. This trips people up constantly:

```
text.toUpperCase();
console.log(text); // still "Hello" – nothing changed!

text = text.toUpperCase(); // you must reassign to keep the result
console.log(text); // "HELLO"
```

b) Accessing Characters

Three ways to grab a character, each with different edge-case behavior — this is the part your notes were specifically testing:

Method	Negative index	Index out of range
<code>text[1]</code>	not supported → <code>undefined</code>	<code>undefined</code>
<code>text.charAt(1)</code>	not supported → "" (empty string)	"" (empty string)
<code>text.at(1) / text.at(-1)</code>	✅ supported — counts from the end	<code>undefined</code>

```
console.log(text[1]);           // "e"
console.log(text.charAt(1));    // "e"
console.log(text.at(-1));       // "o" — last character, easy with at()
console.log(text[-1]);          // undefined — bracket notation can't go backward
```

 **at()** is the newest of the three (ES2022) — it's the only one that supports negative indexing, which is why it's now generally preferred for "give me the last character" type logic.

c) Searching Inside a String

Method	Returns
<code>text.indexOf("l")</code>	index of the first match, or <code>-1</code>
<code>text.lastIndexOf("l")</code>	index of the last match, or <code>-1</code>
<code>text.includes("x")</code>	<code>true/false</code>
<code>text.startsWith("H")</code>	<code>true/false</code> — case-sensitive
<code>text.endsWith("o")</code>	<code>true/false</code> — case-sensitive

```
console.log(text.indexOf("l"));    // 2
console.log(text.lastIndexOf("l")); // 3
console.log(text.includes("x"));    // false
console.log(text.startsWith("h")); // false — "H" ≠ "h", case matters
```

d) Extracting Parts of a String

Your notes tested three overlapping methods — knowing when each behaves differently matters:

Method	Negative start?	Behavior
<code>slice(start, end)</code>	✅ counts from end	<code>end</code> is exclusive
<code>substring(start, end)</code>	❌ treated as 0	<code>end</code> exclusive; swaps args if <code>start > end</code>
<code>substr(start, length)</code>	✅ counts from end	second argument is a length , not an end index

```
console.log(text.slice(-5));        // "Hello" — negative counts back from end
console.log(text.substring(-3, -1)); // "" — negative args clamp to 0, so
```

```
this is substring(0,0)
console.log(text.substr(0, 4)); // "Hell" - 4 characters starting at index 0
```

NEW **Heads up:** `substr()` is **deprecated** in the JavaScript spec (still works everywhere, but official guidance is to avoid it in new code). Prefer `slice()`.

e) Case & Whitespace

```
console.log(text.toUpperCase()); // "HELLO"
console.log(text.toLowerCase()); // "hello"
console.log(" hi ".trimStart()); // "hi "
console.log(" hi ".trimEnd()); // " hi"
console.log(" hi ".trim()); // "hi" NEW - removes both sides at once,
wasn't in your notes but you'll want it constantly
```

f) Combining & Splitting

```
var txt = "world";
console.log(text.concat(" ", txt, " ", "To")); // "Hello world To"
console.log(text.repeat(3)); // "HelloHelloHello"
console.log(text.split(" ")); // ["Hello"] (no space in "Hello" alone)
```

NEW **Modern alternative to `concat()`:** template literals are almost always preferred for building strings now:

```
console.log(`${text} ${txt}`); // "Hello world" - cleaner than concat()
```

12. Array Methods

Setup used throughout this section:

```
var fruits = ['apple', 'orange', 'banana', 'x', 'orange', 'banana'];
```

a) Array Literal vs. `new Array()`

```
var fruits = ['apple', 'orange', 'banana']; // literal - always prefer this
var listObject = new Array('apple', 'orange', 'banana'); // works, but avoid it
```

NEW **Gotcha with `new Array()`:** a single numeric argument doesn't create a one-item array — it sets the *length*:

```
new Array(3);      // [ <3 empty items> ] – an empty array of length 3, NOT [3]
new Array(3, 4);   // [3, 4] – two+ arguments behave normally
```

This inconsistency is exactly why the literal `[]` syntax is the standard.

b) **NEW** Mutating vs. Non-Mutating — the most important distinction

Your notes flagged this in a comment block but didn't explain it — it's one of the most common sources of array bugs. Some methods **change the original array in place**; others **return a brand-new array and leave the original untouched**.

Mutates the original ⚠️	Leaves original unchanged ✅
<code>push</code> , <code>pop</code> , <code>shift</code> , <code>unshift</code>	<code>slice</code> , <code>concat</code> , <code>join</code>
<code>splice</code> , <code>sort</code> , <code>reverse</code>	<code>map</code> , <code>filter</code> , <code>reduce</code>
<code>fill</code> , <code>copyWithin</code>	<code>indexOf</code> , <code>includes</code> , <code>find</code> , <code>every</code> , <code>some</code>

Rule of thumb: if you're not sure, assume it mutates unless you know otherwise — and check the console output of the *original* variable after calling it if you're testing.

c) Adding / Removing from the Ends

Method	Acts on	Returns
<code>push(item)</code>	end — adds	the new length
<code>pop()</code>	end — removes	the removed item
<code>unshift(item)</code>	start — adds	the new length
<code>shift()</code>	start — removes	the removed item

```
console.log(fruits.push("x")); // new length, e.g. 7
console.log(fruits.pop());     // "x" – the item that got removed
console.log(fruits.unshift("x")); // new length again
console.log(fruits.shift());    // "x" – removed item
```

NEW **Correction:** your comment on `shift()` said "return new length" — that's actually `unshift()`'s behavior. `shift()` mirrors `pop()`: it returns the item it removed, not the length. Easy mix-up since the two methods are so symmetrical.

d) `splice()` — Remove, Add, or Replace

`splice(start, deleteCount, ...itemsToInsert)` is the Swiss-army knife of array editing. It **mutates** the array and **returns the removed items**.

```
// Remove: 2 items starting at index 0
console.log(fruits.splice(0, 2)); // removed items, e.g. ["apple", "orange"]
console.log(fruits);              // original array, now shorter

// Add: deleteCount = 0 means nothing removed, only inserted
fruits.splice(1, 0, 2, 3, 4, 5);
console.log(fruits); // new items inserted at index 1

// Replace: remove 3 items starting at index 1, insert 6 and 7 in their place
fruits.splice(1, 3, 6, 7);
console.log(fruits.join(".")); // join() turns the array into a string, joined by
                                // "."
```

e) Combining & Searching

```
var newList = fruits.concat(listObject); // new array, originals untouched

console.log(fruits.indexOf("apple"));    // first matching index, or -1
console.log(fruits.lastIndexOf("banana")); // last matching index, or -1
console.log(fruits.includes("banana"));  // true/false
console.log(fruits.slice(0, 2));         // shallow copy of a portion – does NOT
mutate (unlike splice!)
```

f) Testing & Finding Elements

```
var num = [8, 6, 1, 6, 2];

console.log(num.every(x => x % 2 == 0)); // false – NOT all elements are even
console.log(num.some(x => x % 2 == 0));  // true  – AT LEAST ONE is even
console.log(num.find(x => x % 2 == 0));   // 8    – the first matching VALUE
console.log(num.findIndex(x => x % 2 == 0)); // 0    – the INDEX of that first
match
```

Difference in one line: `find` gives you the element itself; `findIndex` gives you its position.

g) `sort()` — the biggest gotcha in this whole file

```
var num = [8, 6, 1, 6, 2, 200, 100, 101, 110, 120];
console.log(num.sort());
```

You might expect numeric order. What you actually get:

```
[1, 100, 101, 110, 120, 2, 200, 6, 6, 8]
```

Why: by default, `sort()` converts every element to a **string** and compares them alphabetically (character by character), not numerically. "120" comes before "2" alphabetically because '1' < '2', even though 120 > 2 numerically.

The fix — always pass a compare function when sorting numbers:

```
console.log(num.sort((a, b) => a - b)); // ascending: [1, 2, 6, 6, 8, 100, 101, 110, 120, 200]
console.log(num.sort((a, b) => b - a)); // descending
```

This is one of the most common real-world JS bugs — worth memorizing.

h) Removing Duplicates with `Set`

```
var num = [8, 6, 1, 6, 2, 200, 100, 101, 110, 120];
num.sort(); // mutates num in place first (as above)

var uniqueArray = new Set(num);
console.log(uniqueArray); // Set(9) {1, 100, 101, 110, 120, 2, 200, 6, 8} –
duplicate 6 removed
```

A `Set` is a built-in object that only stores **unique** values and preserves insertion order — a quick way to dedupe an array.

 **One catch:** a `Set` is not an array — it doesn't have `.map()`, `.filter()`, etc. To convert it back to an array, use the **spread operator** or `Array.from()`:

```
var uniqueArr = [...uniqueArray]; // spread into a new array
var uniqueArr2 = Array.from(uniqueArray); // equivalent, more explicit
```

i) Transforming Arrays: `map`, `filter`, `reduce`

All three are **non-mutating** — they return something new and leave the original array alone.

```
var num = [8, 6, 1, 6, 2];

console.log(num.map(x => x * 2)); // new array: every element transformed
console.log(num.filter(x => x % 2 == 0)); // new array: only elements that pass the test
```

```
var sum = num.reduce((acc, current) => acc + current, 0);  
console.log(sum); // 23 – boils the whole array down to one value
```

How `reduce` walks through it (matches the trace in your notes):

```
0  + 8 = 8  
8  + 6 = 14  
14 + 1 = 15  
15 + 6 = 21  
21 + 2 = 23
```

`acc` (accumulator) carries the running result forward; `0` is the starting value.  **Note:** your comment wrote the starting value as `intialValue = 0` inside the call — that's not real `reduce` syntax (there's no named parameter there), just a plain third argument: `reduce(callback, 0)`.

j) `reverse()` and Other Mutators

```
console.log(num.reverse()); // flips the array in place – mutates!
```

`fill()` and `copyWithin()` were named in your comment block but not demonstrated — quick reference since you'll meet them eventually:

```
[1, 2, 3, 4].fill(0, 1, 3); // [1, 0, 0, 4] – fills indices 1-2 with 0  
[1, 2, 3, 4, 5].copyWithin(0, 3); // [4, 5, 3, 4, 5] – copies items from index 3  
onward to index 0
```

13. Loops

a) `for` Loop

```
for (start; condition; increment/decrement) {  
  // code  
}
```

```
for (var x = 1; x <= 5; x++) {  
  console.log(x); // 1, 2, 3, 4, 5  
}  
  
for (var x = 5; x > 0; x -= 2) {
```

```
    console.log(x); // 5, 3, 1
}
```

`x++` is shorthand for `x = x + 1`; `x -= 2` is shorthand for `x = x - 2`.

b) **while** Loop — checks the condition **first**

```
var x = 1;
while (x <= 5) {
    console.log(x);
    x++;
}
```

c) **do...while** Loop — runs the body **at least once**, checks after

```
var x = 6;
do {
    console.log(x); // 6 – runs once even though the condition is already false
    x++;
} while (x <= 5);
```

The key difference from while: a regular **while** (`6 <= 5`) would never enter the loop at all, since it checks *before* running. **do...while** guarantees at least one run because it checks *after* the first pass — useful for things like "ask the user for input, then keep asking while it's invalid."

d) **for...of** vs. **for...in**

```
var fruits = ["apple", "orange", "banana"];

for (var x of fruits) {
    console.log(x); // "apple", "orange", "banana" – the VALUES
}

for (var x in fruits) {
    console.log(x);           // "0", "1", "2" – the INDEXES, as STRINGS
    console.log(fruits[x]); // "apple", "orange", "banana" – values via index
}
```

 **Important distinction:** **for...of** gives you values directly and is what you want for arrays. **for...in** gives you keys/indexes (and for arrays, as *strings*, not numbers) — it's really designed for plain objects, not arrays, and is generally **not recommended for arrays** because it also picks up any inherited enumerable properties.

e) **forEach()** — the array-method alternative to a **for** loop

```
fruits.forEach((element, index, arr) => {  
  console.log(element); // current value  
  console.log(index);   // current index  
  console.log(arr);     // the whole array, every time  
});  
  
fruits.forEach(element => console.log(element)); // unused params can be dropped
```

NEW **Arrow function syntax, quickly:** `(element, index, arr) => { ... }` is shorthand for a function expression. Parentheses around the parameter list can be dropped only when there's exactly **one** parameter (`element => ...`); curly braces can be dropped only when the body is a single expression, which is then returned automatically.

NEW **forEach limitation:** unlike a `for` or `for...of` loop, you **cannot** `break` or `continue` out of a `forEach` early — it always runs through the entire array. If you need to stop partway through, use a regular loop instead.

f) Loop Parts Are All Optional

Any of the three `for(...)` slots can be left empty:

```
var count = 0;  
for ( ; count < 5; count++) { // initialization skipped – already declared above  
  console.log(count);  
}  
  
var element = 0;  
for ( ; element < 6; ) {      // increment skipped  
  console.log(element);  
  // ⚠ infinite loop – element never changes, so element < 6 is always true  
}  
  
for ( ; false; ) {  
  console.log("never runs"); // unreachable code – condition is always false  
}  
  
for ( ; true; ) {  
  console.log("runs forever"); // infinite loop – condition is always true  
}  
  
for ( ; ; ) {  
  console.log("also runs forever"); // all 3 parts omitted = condition defaults  
  to true  
}
```

NEW **Note:** `for(;;)` is actually a recognized idiom for "loop on purpose forever" — it's normally paired with a `break` inside the body once some condition is met, rather than left to run unchecked.

g) The Hoisting Gotcha — why one version fails silently and the other crashes

```
var item;
for ( ; item < 5; item++) {
  console.log(item);
}
// loop body never runs – no error, no output

for ( ; x < 5; x++) {
  console.log(x);
}
// ReferenceError: x is not defined
```

Why the difference: `var item;` declares the variable (even with no value), so `item` exists and is `undefined`. Comparing `undefined < 5` coerces `undefined` to `NaN` (see Section 8), and any comparison against `NaN` is `false` — so the loop simply never starts. But `x` was never declared anywhere with `var/let/const`, so JavaScript has nothing to fall back on — referencing it throws immediately.

This is also a preview of why `let/const` (still on your to-learn list — see the closing section) are considered safer than `var`: `var` declarations are hoisted to the top of their scope and quietly filled with `undefined`, which is exactly what let a bug like this run without error instead of crashing where the mistake actually is.

14. Conditional Statements

a) `if / else if / else`

```
var age = 20;
if (age >= 18) {
  console.log("Adult");
}

var grade = 20;
if (grade > 50 && grade <= 65) {
  console.log("pass");
} else if (grade > 65 && grade <= 75) {
  console.log("good");
} else if (grade > 75 && grade <= 85) {
  console.log("very good");
} else if (grade > 85) {
  console.log("Excellent");
} else {
  console.log("fail"); // grade = 20 falls through every condition above → "fail"
}
```

b) Logical Operators — the mechanics behind `&&`

Operator	Name	True when
&&	AND	both sides are true
	OR	at least one side is true
!	NOT	flips a boolean

```
console.log(true && false); // false
console.log(true || false); // true
console.log(!true);         // false
```

Short-circuiting: `&&` stops and returns the first falsy value it hits without checking the rest; `||` stops and returns the first truthy value. This is why `grade > 50 && grade <= 65` is efficient and safe — if the first part is false, JS doesn't even bother evaluating the second.

c) `switch` Statement

```
var num = 1;
switch (num) {
  case 4:
    console.log("num is 4");
    break;
  case 3:
    console.log("num is 3");
    break;
  case 5:
    console.log("num is 5");
    break;
  default:
    console.log("num is not in these");
}
```

Matching uses strict equality (`===`) between the switch value and each `case`.

Fallthrough — what `break` is actually preventing:

```
switch (2) {
  case 1:
    console.log("one");
  case 2:
    console.log("two"); // matches here
  case 3:
    console.log("three"); // no break above → runs anyway
    break; // stops here
  case 4:
    console.log("four");
}
```

```
}  
// Output: "two" then "three"
```

Without **break**, execution doesn't stop at the matching case — it keeps running every case after it until it hits a **break** or the end of the switch. This is called **fallthrough**, and it's almost always accidental when it happens by mistake — but occasionally used on purpose to group cases that should share behavior.

d) Ternary Operator — a compact **if/else**

```
x > 0 ? console.log("Greater Than") : console.log("less Than");
```

 **More idiomatic use:** the ternary is an *expression* — it produces a value — so it's normally used to assign a result rather than to pick which side-effect to run:

```
var result = x > 0 ? "Greater Than" : "less Than";  
console.log(result);
```

e) Combining Ternaries with **switch**

Your notes used a nested ternary to compute a switch value (a sign function: -1, 0, or 1):

```
var x = 1;  
switch (x > 0 ? 1 : (x < 0 ? -1 : 0)) {  
  case 1:  
    console.log("positive");  
    break;  
  case -1:  
    console.log("negative");  
    break;  
  default:  
    console.log("0");  
    break;  
}  
// x = 1 → x > 0 is true → expression evaluates to 1 → matches case 1 → "positive"
```

And the **switch(true)** pattern turns **switch** into something that behaves like an **if/else if** chain, by making every **case** a boolean expression:

```
var x = -1;  
switch (true) {  
  case x > 0:  
    console.log("positive");  
    break;  
  case x < 0:
```

```
        console.log("negative"); // x = -1, so x < 0 is true → this runs
        break;
    default:
        console.log("0");
        break;
}
```

This works because `switch` compares its value (`true`) against each `case` expression's result — so the first `case` that evaluates to `true` is the one that matches.

15. Functions

a) Declaring & Calling

```
function sum(num1, num2) {
    var total = num1 + num2;
    console.log(total);
}
sum(3, 4); // 7
```

b) What Happens with Missing Arguments

```
sum(3); // only one argument passed
// num2 is undefined
// total = 3 + undefined = NaN
```

JavaScript never enforces how many arguments you pass — extra ones are ignored, missing ones become `undefined`. Any arithmetic involving `undefined` produces `NaN` (same rule as Section 8).

c) Rest Parameter (`...`) — accept any number of arguments

```
function sum(...numbers) {
    var total = 0;
    for (var num of numbers) {
        total += num;
    }
    console.log(total);
    return total;
}
var x = sum(3, 4, 5);
console.log(x); // 12

function mul(...numbers) {
    var total = 1; // starts at 1, not 0 – the "do-nothing" value for
    multiplication
}
```

```
    for (var num of numbers) {  
        total *= num;  
    }  
    console.log(total);  
    return total;  
}  
mul(2, 3, 5); // 30
```

`...numbers` (the **rest parameter**) gathers every argument passed into the function into one real array called `numbers` — no matter whether you pass 0, 3, or 100 values. This solves the Section (b) problem above: there's no fixed number of parameters to fall short of.

 **Connecting the dots:** this is the same `...` syntax flagged as missing back when discussing `Set` → array conversion — but running in the *opposite direction*:

- **Rest** (here): gathers individual arguments **into** an array — `function sum(...numbers)`.
- **Spread** (still to come): expands an array **out into** individual values — `Math.max(...numbers)` or `[...arr1, ...arr2]`.

d) `return` vs. `console.log` Inside a Function

`console.log` inside a function only prints to the console — it's a side effect for you to look at while debugging. `return` is what actually hands a value back to whatever called the function, so it can be stored, reused, or passed elsewhere:

```
var x = sum(3, 4, 5); // works because sum() has `return total;`  
console.log(x);      // 12  
  
// If sum() only console.log'd and never returned:  
// var x = sum(3, 4, 5); would make x equal to undefined,  
// even though "12" was printed to the console.
```

What I Added (things missing from the session that you'll need soon)

Most gaps from this batch are now woven directly into Sections 13–15 (the `do...while` vs `while` distinction, `for...of` vs `for...in`, why `forEach` can't `break`, switch fallthrough, the `switch(true)` pattern, and what happens when a function is called with too few arguments). A few bigger topics are still worth flagging separately:

1. **let and const** — Still `var`-only in every file so far, and now more relevant than ever: Section 13g shows a real bug (an undeclared loop caused a silent no-op instead of a crash) that's a direct consequence of `var`'s hoisting behavior. `let/const` don't hoist the same way, which would have surfaced that mistake immediately instead of it running quietly with no output.
2. **Template literals** — backtick strings with embedded expressions, previewed in Section 11f.

3. **Primitive vs. Reference types** — still the underlying reason `{ } == { }` is `false`, and also explains why the `arr` parameter inside `forEach((element, index, arr) => ...)` is the *same* array in memory, not a copy.
4. **Spread operator** — now half-covered: Section 15c showed the **rest** parameter (`...numbers` gathering arguments into an array). Its mirror image, **spread** (expanding an array back out — `Math.max(...numbers), [...arr1, ...arr2]`), still hasn't shown up in your notes yet.
5. **Regex-based string methods** — `replace()`, `replaceAll()`, `match()` — still not covered.

Want me to build a follow-up guide covering these, in the same style?